Sesión 20: BLoC: Implementación y Práctica (Uso de BlocProvider, BlocBuilder, BlocListener y BlocConsumer)

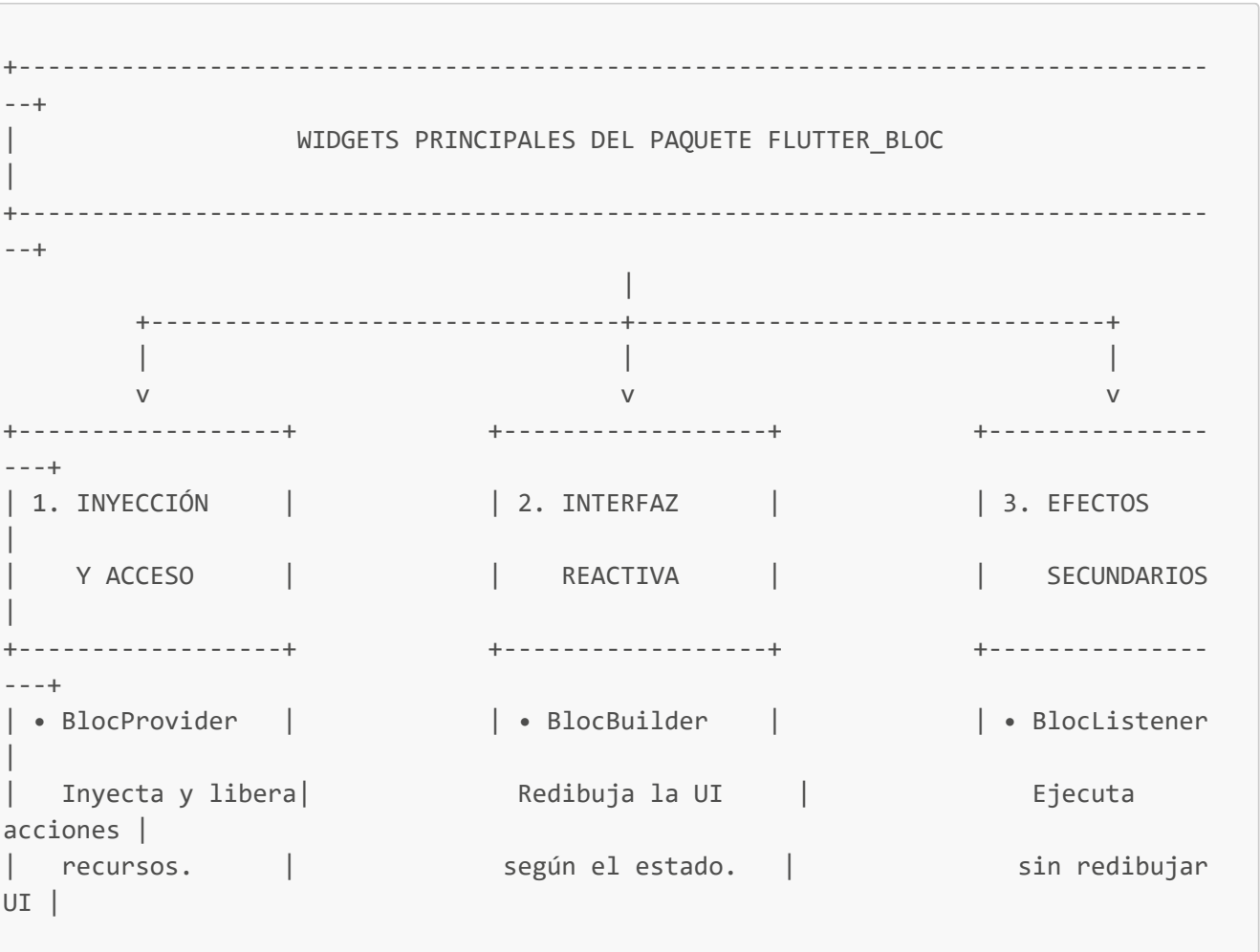
Introducción

En la sesión anterior establecimos los principios teóricos del patrón BLoC y Cubit, comprendiendo cómo los flujos de eventos y estados garantizan el desacoplamiento entre la interfaz de usuario y la lógica de negocio. Para llevar esta arquitectura a la práctica, el paquete `flutter_bloc` ofrece una serie de widgets especializados diseñados para gestionar el ciclo de vida de los BLoCs, inyectarlos en el árbol de componentes y reaccionar de forma eficiente a sus emisiones.

En esta sesión profundizaremos en la implementación práctica de los componentes fundamentales de `flutter_bloc`: `BlocProvider`, `BlocBuilder`, `BlocListener` y `BlocConsumer`. Analizaremos sus casos de uso específicos, sus parámetros de optimización como `buildWhen` y `listenWhen`, y cómo estructurar una pantalla completa que reaccione tanto a cambios gráficos como a efectos secundarios en tiempo real.

Conceptos y Definiciones

La integración fluida entre Flutter y BLoC se logra mediante cuatro widgets clave que actúan como puente entre la capa de presentación y la lógica reactiva.



```
| • MultiBlocProvider|          | • buildWhen      |
(SnackBars/Nav). |
|   Múltiples BLoCs|            Filtra re-renders  |            | •
BlocConsumer   |
|   en paralelo.   |            innecesarios.      |              Integra
Builder  |
+------------------+          +------------------+              y Listener.
|
                                                              +---------------
---+
```

1. Inyección de Dependencias: `BlocProvider` y `MultiBlocProvider`

- **`BlocProvider`:**
- Inyecta una instancia de BLoC/Cubit en el árbol de widgets utilizando `InheritedWidget` de forma subyacente.
- Encargado de la gestión del ciclo de vida: crea el BLoC mediante su propiedad `create` y lo destruye automáticamente (`close()`) cuando el widget sale del árbol.
- Permite acceder a la instancia en los hijos mediante `BlocProvider.of<MiBloc>(context)` o la extensión `context.read<MiBloc>()`.
- **`MultiBlocProvider`:** Evita el anidamiento excesivo (*pyramid of doom*) cuando una vista requiere múltiples BLoCs o Cubits simultáneamente.

---

2. Renderizado Reactivo: `BlocBuilder` y la propiedad `buildWhen`

- **`BlocBuilder<B, S>`:**
- Escucha las emisiones del BLoC de tipo `B` con estado de tipo `S` y ejecuta su función `builder(context, state)` para reconstruir la interfaz.
- **Optimización con `buildWhen`:**
- Recibe el estado anterior (`previous`) y el estado actual (`current`), retornando un booleano.
- Si evalúa a `true`, ejecuta el `builder`; si evalúa a `false`, ignora el cambio y evita re-renders innecesarios en la interfaz.

---

3. Efectos Secundarios y Combinación: `BlocListener` y `BlocConsumer`

- **`BlocListener<B, S>`:**
- Diseñado exclusivamente para reaccionar a cambios de estado ejecutando acciones puntuales ("efectos puros") como mostrar `SnackBar`, `AlertDialog` o realizar una navegación (`Navigator.push`).
- **No** redibuja la interfaz de usuario.

- Incluye la propiedad `listenWhen` para filtrar cuándo debe dispararse el callback `listener(context, state)`.
- **`BlocConsumer<B, S>`:**
- Combina `BlocBuilder` y `BlocListener` en un solo widget cuando una vista necesita simultáneamente redibujar componentes y disparar efectos secundarios ante el mismo cambio de estado.

Ejemplo Completo con Código Fuente: Sistema de Descarga de Archivos con BLoC Practico

El siguiente ejemplo simula un gestor de descarga de archivos académicos en ESCOM. Demuestra el uso combinado de `MultiBlocProvider`, `BlocBuilder` con filtro `buildWhen`, y `BlocConsumer` para mostrar avisos y actualizar la barra de progreso reactivamente.

> **Nota de dependencia:** Se requiere incluir `flutter_bloc: ^8.1.3` y `equatable: ^2.0.5` en el archivo `pubspec.yaml`.

Código Fuente Explicado (`lib/main.dart`)

```dart
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:equatable/equatable.dart';

void main() {
  runApp(const DemoPracticaBlocApp());
}

// ============================================================================
// 1. DEFINICIÓN DE EVENTOS Y ESTADOS
// ============================================================================

abstract class DescargaEvent extends Equatable {
  const DescargaEvent();
  @override
  List<Object?> get props => [];
}

class IniciarDescargaEvent extends DescargaEvent {
  final String nombreArchivo;
  const IniciarDescargaEvent(this.nombreArchivo);

  @override
  List<Object?> get props => [nombreArchivo];
}

class CancelarDescargaEvent extends DescargaEvent {}

class _ProgresoActualizadoEvent extends DescargaEvent {
  final int progreso;
```

```
  const _ProgresoActualizadoEvent(this.progreso);

  @override
  List<Object?> get props => [progreso];
}

// --- ESTADOS ---
abstract class DescargaState extends Equatable {
  const DescargaState();
  @override
  List<Object?> get props => [];
}

class DescargaInicialState extends DescargaState {}

class DescargaEnProgresoState extends DescargaState {
  final String nombreArchivo;
  final int progreso; // 0 a 100

  const DescargaEnProgresoState({
    required this.nombreArchivo,
    required this.progreso,
  });

  @override
  List<Object?> get props => [nombreArchivo, progreso];
}

class DescargaCompletadaState extends DescargaState {
  final String nombreArchivo;
  const DescargaCompletadaState(this.nombreArchivo);

  @override
  List<Object?> get props => [nombreArchivo];
}

class DescargaErrorState extends DescargaState {
  final String mensaje;
  const DescargaErrorState(this.mensaje);

  @override
  List<Object?> get props => [mensaje];
}

// ============================================================================
// 2. BLOC IMPLEMENTACIÓN
// ============================================================================

class DescargaBloc extends Bloc<DescargaEvent, DescargaState> {
  StreamSubscription<int>? _descargaSubscription;

  DescargaBloc() : super(DescargaInicialState()) {
    on<IniciarDescargaEvent>(_onIniciarDescarga);
    on<CancelarDescargaEvent>(_onCancelarDescarga);
```

```
    on<_ProgresoActualizadoEvent>(_onProgresoActualizado);
  }

  Future<void> _onIniciarDescarga(
    IniciarDescargaEvent event,
    Emitter<DescargaState> emit,
  ) async {
    emit(DescargaEnProgresoState(nombreArchivo: event.nombreArchivo, progreso:
0));
    await _descargaSubscription?.cancel();

    // Simulación de descarga asíncrona por bloques mediante Stream
    _descargaSubscription = Stream<int>.periodic(
      const Duration(milliseconds: 300),
      (i) => (i + 1) * 20,
    ).take(5).listen((progreso) {
      add(_ProgresoActualizadoEvent(progreso));
    });
  }

  void _onProgresoActualizado(
    _ProgresoActualizadoEvent event,
    Emitter<DescargaState> emit,
  ) {
    if (state is DescargaEnProgresoState) {
      final estadoActual = state as DescargaEnProgresoState;
      if (event.progreso >= 100) {
        emit(DescargaCompletadaState(estadoActual.nombreArchivo));
      } else {
        emit(DescargaEnProgresoState(
          nombreArchivo: estadoActual.nombreArchivo,
          progreso: event.progreso,
        ));
      }
    }
  }

  void _onCancelarDescarga(
    CancelarDescargaEvent event,
    Emitter<DescargaState> emit,
  ) {
    _descargaSubscription?.cancel();
    emit(DescargaInicialState());
  }

  @override
  Future<void> close() {
    _descargaSubscription?.cancel();
    return super.close();
  }
}

// ============================================================================
// 3. INTERFAZ DE USUARIO CON IMPLEMENTACIÓN PRÁCTICA DE BLOC
```

```dart
// ============================================================================

class DemoPracticaBlocApp extends StatelessWidget {
  const DemoPracticaBlocApp({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocProvider<DescargaBloc>(
      create: (context) => DescargaBloc(),
      child: MaterialApp(
        title: 'ESCOM - Práctica BLoC',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          useMaterial3: true,
          colorScheme: ColorScheme.fromSeed(
            seedColor: const Color(0xFF6B112D), // Guinda Institucional
          ),
        ),
        home: const PantallaGestorDescargas(),
      ),
    );
  }
}

class PantallaGestorDescargas extends StatelessWidget {
  const PantallaGestorDescargas({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Gestor de Descargas ESCOM'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),
      body: Padding(
        padding: const EdgeInsets.all(20.0),
        child: Column(
          children: [
            const Text(
              'Descarga de Material Didáctico',
              style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 20),

            // USO DE BLOCCONSUMER: Maneja UI y Efectos Secundarios al mismo
tiempo
            BlocConsumer<DescargaBloc, DescargaState>(
              listener: (context, state) {
                if (state is DescargaCompletadaState) {
                  ScaffoldMessenger.of(context).showSnackBar(
                    SnackBar(
                      content: Text('¡Descarga finalizada:
${state.nombreArchivo}!'),
```

```
                        backgroundColor: Colors.green,
                      ),
                    );
                  } else if (state is DescargaErrorState) {
                    ScaffoldMessenger.of(context).showSnackBar(
                      SnackBar(
                        content: Text('Error: ${state.mensaje}'),
                        backgroundColor: Colors.red,
                      ),
                    );
                  }
                },
                builder: (context, state) {
                  if (state is DescargaEnProgresoState) {
                    return Column(
                      children: [
                        Text('Descargando: ${state.nombreArchivo}'),
                        const SizedBox(height: 12),
                        LinearProgressIndicator(value: state.progreso / 100),
                        const SizedBox(height: 8),
                        Text('${state.progreso}%'),
                        const SizedBox(height: 12),
                        ElevatedButton.icon(
                          onPressed: () {
                            context.read<DescargaBloc>
().add(CancelarDescargaEvent());
                          },
                          icon: const Icon(Icons.cancel),
                          label: const Text('Cancelar'),
                          style: ElevatedButton.styleFrom(
                            foregroundColor: Colors.red,
                          ),
                        ),
                      ],
                    );
                  }

                  if (state is DescargaCompletadaState) {
                    return Card(
                      color: Colors.green.shade50,
                      child: ListTile(
                        leading: const Icon(Icons.check_circle, color:
Colors.green),
                        title: Text(state.nombreArchivo),
                        subtitle: const Text('Archivo listo para abrir'),
                        trailing: IconButton(
                          icon: const Icon(Icons.refresh),
                          onPressed: () {
                            context.read<DescargaBloc>
().add(CancelarDescargaEvent());
                          },
                        ),
                      ),
                    );
```

```
                }

                // Estado por defecto: DescargaInicialState
                return Card(
                  child: ListTile(
                    leading: const Icon(Icons.picture_as_pdf, color: Colors.red),
                    title: const Text('Guia_Practica_Flutter_BloC.pdf'),
                    subtitle: const Text('Tamaño: 12.4 MB'),
                    trailing: ElevatedButton(
                      onPressed: () {
                        context.read<DescargaBloc>().add(
                              const IniciarDescargaEvent(
                                'Guia_Practica_Flutter_BloC.pdf',
                              ),
                            );
                      },
                      child: const Text('Descargar'),
                    ),
                  ),
                );
              },
            ),

            const SizedBox(height: 30),
            const Divider(),
            const SizedBox(height: 10),

            // USO DE BLOCBUILDER CON OPTIMIZACIÓN buildWhen
            const Text(
              'Monitor Optimizado (Solo actualiza en progreso)',
              style: TextStyle(fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 10),
            BlocBuilder<DescargaBloc, DescargaState>(
              buildWhen: (previous, current) {
                // Se redibuja ÚNICAMENTE cuando el estado cambia entre progresos
                return current is DescargaEnProgresoState;
              },
              builder: (context, state) {
                if (state is DescargaEnProgresoState) {
                  return Text(
                    'Registro de Telemetría: Carga al ${state.progreso}%',
                    style: const TextStyle(color: Colors.grey),
                  );
                }
                return const Text('Telemetría en espera...');
              },
            ),
          ],
        ),
      ),
    );
  }
```

```
}
```

Prácticas Guiadas / Laboratorio

Práctica 1: Implementación de Optimización en UI con `buildWhen`

1. Modifica la vista del laboratorio agregando un contador de reconstrucciones mediante una variable local en el método `build`.
2. Implementa la propiedad `buildWhen` en un `BlocBuilder` para que ignore las actualizaciones si el porcentaje de descarga no ha cambiado en al menos un 40%.
3. Verifica en la consola de depuración cómo se reduce drásticamente el número de re-renderizados de la vista gracias al filtro condicional.

Práctica 2: Manejo de Diálogos de Confirmación con `BlocListener`

1. Añade un estado `DescargaConfirmacionRequeridaState` al BLoC.
2. Utiliza un `BlocListener` para detectar este estado y desplegar un diálogo `showDialog()` preguntando al usuario si desea sobrescribir el archivo existente.
3. Envía los eventos correspondientes desde los botones "Aceptar" o "Cancelar" del diálogo para reanudar o abortar el flujo del BLoC.

Resumen y Puntos Clave

- **`BlocProvider`:** Administra el ciclo de vida e inyecta la instancia del BLoC/Cubit mediante el árbol de widgets.
- **`BlocBuilder`:** Reconstruye la interfaz ante cambios de estado; se optimiza mediante la propiedad `buildWhen`.
- **`BlocListener`:** Ejecuta efectos secundarios puros (navegación, alertas, snackbars) sin provocar renderizados gráficos; se condiciona con `listenWhen`.
- **`BlocConsumer`:** Integra las capacidades de `BlocBuilder` y `BlocListener` en un solo componente cuando la vista requiere actualizar gráficos y disparar efectos simultáneamente.

Bibliografía y Referencias

1. **Angelov, F.** (2024). *Flutter Bloc Widgets Documentation*. Recuperado de: https://bloclibrary.dev/flutter-bloc-concepts/
2. **Google LLC.** (2024). *Architecting Flutter Applications with BLoC*. Recuperado de: https://docs.flutter.dev/data-and-backend/state-mgmt/options#bloc--rx
3. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.